

شبیه‌سازی و تحلیل عملکرد پروتکل‌های کنترل جریان لایه data link گزارش پروژه درس شبکه‌های مخابراتی - تکلیف اول

نگارنده: سیاوش ابراهیمی کمال (شماره دانشجویی: ۴۰۲۱۴۳۳۳)

استاد درس: جناب آقای دکتر جمشیدی

فهرست مطالب

۱	مقدمه و اهداف پروژه	۲
۲	مبانی تئوری و روابط ریاضی سیستم	۳
۱.۲	فرموله‌بندی شاخص‌های ارزیابی عملکرد	۳
۲.۲	معادلات بهره‌وری تئوریک پروتکل‌ها	۳
۳	معماری سیستم و تشریح کدهای شبیه‌سازی (C++)	۴
۱.۳	تشریح خط‌به‌خط کد پروتکل Stop-and-Wait	۴
۲.۳	تشریح پیاده‌سازی پروتکل Go-Back-N (فایل hw1-gbn.cc)	۱۰
۳.۳	پروتکل Selective Repeat (فایل hw1-sr.cc)	۱۴
۴	اتوماسیون داده‌گیری و کد تحلیل پایتون	۱۹
۵	تحلیل مهندسی خروجی‌ها و بررسی نمودارها	۲۰
۱.۵	سری اول: ارزیابی شبکه تحت تغییرات نرخ خطای بیت (BER)	۲۰
۲.۵	سری دوم: ارزیابی شبکه تحت تغییرات تاخیر انتشار (Propagation Delay)	۲۲
۳.۵	سری سوم: ارزیابی شبکه تحت تغییرات ابعاد پنجره لغزنده (Window Size)	۲۴
۶	نتیجه‌گیری نهایی و دستاوردهای مهندسی	۲۵

۱ مقدمه و اهداف پروژه

مدیریت کارآمد منابع شبکه و تضمین انتقال بدون خطای داده‌ها از وظایف کلیدی لایه data link در سیستم‌های مخابراتی مدرن است. در این پروژه، به شبیه‌سازی، ارزیابی ساختاریافته و تحلیل عملکرد سه پروتکل بنیادی کنترل جریان و بازگرداندن خودکار خطا (ARQ) یعنی پروتکل‌های Stop-and-Wait، Go-Back-N (GBN) و Selective Repeat (SR) پرداخته شده است. هدف اصلی این پروژه، ارزیابی رفتار فیزیکی این سه مکانیزم تحت فشارهای مختلف کانال مخابراتی است. شبیه‌سازی‌ها در بستری اتوماتیک با ابزار قدرتمند ns-3 Discrete-Event Network Simulator پیاده‌سازی شده‌اند. در این ارزیابی، سه متغیر مستقل فیزیکی شامل:

۱. نرخ خطای بیت (BER): از بازه کانال ایده‌آل (0) تا نویز شدید (10^{-3}).

۲. تاخیر انتشار کانال (Propagation Delay): از کانال‌های محلی (1ms) تا لینک‌های دوربرد زمان‌بر (100ms).

۳. اندازه پنجره فرستنده و گیرنده (N و W): از پنجره تک‌بسته‌ای تا ابعاد بزرگ (256).

بر روی چهار پارامتر عملکردی کلیدی (خروجی‌های آزمایش) شامل توان عملیاتی مفید (Goodput)، میانگین تاخیر بسته (Average Delay)، نرخ بهره‌وری لینک (Link Utilization) و تعداد ارسال‌های مجدد (Retransmissions) بررسی شده و توازن‌های مهندسی (Trade-offs) هر یک استخراج گردیده است.

۲ مبانی تئوری و روابط ریاضی سیستم

برای درک بهتر خروجی‌های شبیه‌ساز ns-3، لازم است شاخص‌های عملکردی شبکه را فرموله‌بندی کنیم. ظرفیت لینک مخابراتی ما در این پروژه ثابت و برابر با $C = 5\text{Mbps}$ تنظیم شده است و ابعاد بسته‌ها $L = 1024\text{Bytes}$ است.

۱.۲ فرموله‌بندی شاخص‌های ارزیابی عملکرد

۱. توان عملیاتی مفید (**Goodput**): نرخ بیت‌های مفیدی است که به طور صحیح توسط لایه اپلیکیشن گیرنده تحویل گرفته شده است:

$$\text{Goodput (bps)} = \frac{\text{Bytes Received Valid Total} \times 8}{\text{Time Simulation Active}} \quad (۱)$$

۲. بهره‌وری لینک (**Link Utilization**): نسبت زمان اشغال بودن لینک توسط دیتای ارسالی (شامل بسته‌های اصلی و بسته‌های تکراری ناشی از خطا) به کل زمان در دسترس کانال است:

$$\text{Utilization (\%)} = \frac{\text{retransmissions (including Bytes Sent Total)} \times 8}{C \times \text{Time Simulation Active}} \times 100 \quad (۲)$$

۳. میانگین تاخیر زمان انتقال (**Average End-to-End Delay**): میانگین زمان سپری شده از لحظه تولید و ارسال بسته در لایه پیوند داده فرستنده تا لحظه دریافت کامل آن در لایه بالای گیرنده:

$$\text{Delay Average (s)} = \frac{\sum_{i=1}^M (T_{\text{arrival},i} - T_{\text{sent},i})}{M} \quad (۳)$$

که در آن M تعداد کل بسته‌های به مقصد رسیده است.

۲.۲ معادلات بهره‌وری تئوریک پروتکل‌ها

بهره‌وری لینک وابسته به پارامتر بدون بعد $a = \frac{T_{\text{prop}}}{T_{\text{trans}}}$ تعریف می‌شود که در آن $T_{\text{trans}} = \frac{L}{C}$ زمان ارسال یک بسته روی خط است.

• پروتکل **Stop-and-Wait**: فرستنده پس از ارسال یک بسته متوقف شده و منتظر ACK می‌ماند، بنابراین در حالت ایده‌آل بدون خطا:

$$U_{\text{sw}} = \frac{1}{1 + 2a} \quad (۴)$$

• پروتکل **Go-Back-N**: با استفاده از پنجره‌ای به حجم N ، فرستنده کابل را پر می‌کند. اگر پنجره بتواند تاخیر رفت و برگشت را پوشش دهد ($N > 1 + 2a$)، بهره‌وری به 100% متمایل می‌شود؛ در غیر این صورت لینک خالی می‌ماند:

$$U_{\text{GBN}} = \begin{cases} 1 & \text{if } N \geq 1 + 2a \\ \frac{N}{1 + 2a} & \text{if } N < 1 + 2a \end{cases} \quad (۵)$$

• پروتکل **Selective Repeat**: همانند GBN عمل می‌کند، با این تفاوت که در حضور خطای کانال با احتمال خطای بسته $P_f = 1 - (1 - \text{BER})^L$ ، جریمه کمتری دارد. بهره‌وری تئوریک آن در محیط نویزی به این صورت مدل می‌شود:

$$U_{\text{SR}} = (1 - P_f) \times \min \left(1, \frac{W}{1 + 2a} \right) \quad (۶)$$

۳ معماری سیستم و تشریح کدهای شبیه‌سازی (C++)

برای پیاده‌سازی متدولوژی‌های فوق، سه اسکریپت شبیه‌سازی مستقل ساختاریافته در فریم‌ورک ns-3 توسعه داده شده است. برنامه با شبیه‌سازی یک لینک نقطه‌به‌نقطه (Point-to-Point) بین دو گره فرستنده (Node 0) و گیرنده (Node 1) پیکربندی شده و خطای بیت با شیء RateErrorModel به صورت پویا اعمال می‌گردد. در هر سه برنامه، برای بررسی دقیق تر رفتار سیستم در شرایط اشباع خط، حجم کل بسته‌های آماده برای ارسال به 10,000 افزایش یافته است.

۱.۳ تشریح خطبه‌خط کد پروتکل Stop-and-Wait

بخش اول: پیکربندی ماژول‌ها و کتابخانه‌های پایه

```
scratch > hw1-sw.cc
1  #include "ns3/core-module.h"
2  #include "ns3/network-module.h"
3  #include "ns3/internet-module.h"
4  #include "ns3/applications-module.h"
5  #include "ns3/point-to-point-module.h"
6  #include "ns3/seq-ts-header.h"
7  #include <fstream>
8  #include <vector>
9  #include <string>
10
11  using namespace ns3;
12
```

شکل ۱: فراخوانی ماژول‌های ns-3 و کتابخانه‌های استاندارد C++

در ابتدای پیاده‌سازی این پروتکل، ماژول‌های اساسی شبیه‌ساز ns-3 شامل هسته شبیه‌ساز، تنظیمات شبکه، پروتکل اینترنت و لینک‌های نقطه‌به‌نقطه (Point-to-Point) را وارد کردم. هدر seq-ts-header.h به من اجازه می‌دهد تا شماره توالی و برچسب زمانی را به هر بسته اضافه کرده و تاخیر انتقال را درگیرنده محاسبه کنم. همچنین کتابخانه‌های استاندارد C++ مانند fstream (برای خروجی CSV)، vector و string را برای مدیریت و ذخیره داده‌ها اضافه کردم. این بخش ساختاری در پروتکل‌های SR و GBN نیز کاملاً مشترک است.

بخش دوم: تعریف ساختار داده خروجی

```
struct SimResult {
    double goodput;
    double avgDelay;
    double utilization;
    uint32_t retransmissions;
};
```

شکل ۲: تعریف ساختار SimResult برای ذخیره نتایج

برای کپسوله‌سازی و استخراج مرتب نتایج شبیه‌سازی، ساختار داده‌ای به نام SimResult را تعریف کردم. این ساختار چهار پارامتر کلیدی پروژه یعنی توان عملیاتی مفید (goodput)، میانگین تاخیر (avgDelay)، بهره‌وری لینک (utilization) و تعداد ارسال‌های مجدد (retransmissions) را در یک بسته واحد جمع‌آوری می‌کند. با این کار، بازگرداندن نتایج از تابع شبیه‌ساز و ثبت یکپارچه آن‌ها در فایل‌های CSV بسیار ساده‌تر انجام می‌شود. این ساختار نیز در تمامی پروتکل‌های پیاده‌سازی شده مشترک است.

بخش سوم: تعریف کلاس فرستنده (StopAndWaitSender)

```
20 class StopAndWaitSender : public Application
21 {
22 public:
23     StopAndWaitSender ();
24     virtual ~StopAndWaitSender ();
25     void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout);
26     uint32_t GetTotalBytesSent () const { return m_totalBytesSent; }
27 private:
28     virtual void StartApplication (void) override;
29     virtual void StopApplication (void) override;
30
31     void SendPacket ();
32     void ReceiveAck (Ptr<Socket> socket);
33     void HandleTimeout ();
34
35     Ptr<Socket>      m_socket;
36     Address          m_peerAddress;
37     uint32_t         m_packetSize;
38     uint32_t         m_nPackets;
39     Time             m_timeout;
40
41     uint32_t         m_packetsSent;
42     uint8_t          m_seqNo;
43     EventId          m_timeoutEvent;
44     Ptr<Packet>      m_unackedPacket;
45
46     uint32_t         m_totalBytesSent;
47 };
48
```

شکل ۳: تعریف ساختار کلاس فرستنده و متغیرهای حالت

در این بخش، کلاس فرستنده یعنی StopAndWaitSender را با ارث‌بری از کلاس پایه Application در ns-3 ایجاد کردم. در بخش public، علاوه بر سازنده و مخرب، تابع Setup را برای مقداردهی اولیه پارامترهایی مثل سوکت، آدرس مقصد و ابعاد بسته قرار دادم.

در بخش private، منطق اصلی پروتکل را در قالب توابعی برای ارسال بسته (SendPacket)، دریافت تاییدیه (ReceiveAck) و مدیریت انقضای زمان (HandleTimeout) تعریف کردم. همچنین متغیرهای حیاتی این پروتکل مانند رویداد زمان‌سنج (m-timeoutEvent)، کپی بسته ارسال‌شده برای بازپخش احتمالی (m-unackedPacket) و شماره توالی صفر و یک (m-seqNo) را در همین قسمت تعریف کرده‌ام تا وضعیت فرستنده در هر لحظه حفظ شود.

بخش چهارم: پیاده‌سازی توابع پایه‌ای فرستنده

```
49
50 StopAndWaitSender::StopAndWaitSender () : m_packetsSent(0), m_seqNo(0), m_totalBytesSent(0) {}
51 StopAndWaitSender::~StopAndWaitSender () { m_socket = 0; }
52
53 void StopAndWaitSender::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout)
54 {
55     m_socket = socket; m_peerAddress = address; m_packetSize = packetSize;
56     m_nPackets = nPackets; m_timeout = timeout;
57 }
58
59 void StopAndWaitSender::StartApplication (void)
60 {
61     m_socket->Bind ();
62     m_socket->Connect (m_peerAddress);
63     m_socket->SetRecvCallback (MakeCallback (&StopAndWaitSender::ReceiveAck, this));
64     SendPacket ();
65 }
66
67 void StopAndWaitSender::StopApplication (void)
68 {
69     if (m_socket) { m_socket->Close (); }
70     Simulator::Cancel (m_timeoutEvent);
71 }

```

شکل ۴: پیاده‌سازی توابع راه‌اندازی و توقف در کلاس فرستنده

در این بخش، توابع اولیه و مدیریتی فرستنده را پیاده‌سازی کردم. در سازنده کلاس، شمارنده‌های اولیه را صفر کرده و در تابع Setup مقادیر ورودی را به متغیرهای کلاس اختصاص دادم. در تابع StartApplication که در لحظه شروع برنامه اجرا می‌شود، سوکت را به گره مقصد متصل کرده و تابع ReceiveAck را به عنوان تابع دریافت‌کننده (Callback) تنظیم کردم؛ سپس با فراخوانی SendPacket فرایند ارسال را کلید زدم. در نهایت، در تابع StopApplication سوکت را مسدود کرده و رویدادهای زمان‌سنج را لغو کردم تا برنامه بدون خطا خاتمه یابد.

```

73 void StopAndWaitSender::SendPacket ()
74 {
75     Ptr<Packet> packet = Create<Packet> (m_packetSize);
76
77     SeqTsHeader seqTs;
78     seqTs.SetSeq (m_seqNo);
79     packet->AddHeader (seqTs);
80
81     m_unackedPacket = packet->Copy ();
82     m_socket->Send (packet);
83
84     m_totalBytesSent += packet->GetSize ();
85     m_timeoutEvent = Simulator::Schedule (m_timeout, &StopAndWaitSender::HandleTimeout, this);
86 }
87
88 void StopAndWaitSender::HandleTimeout ()
89 {
90     m_socket->Send (m_unackedPacket);
91     m_totalBytesSent += m_unackedPacket->GetSize ();
92     m_timeoutEvent = Simulator::Schedule (m_timeout, &StopAndWaitSender::HandleTimeout, this);
93 }

```

شکل ۵: پیاده‌سازی مکانیزم ارسال اولیه و بازپخش بسته در صورت Timeout

در تابع `SendPacket`، ابتدا یک بسته جدید ایجاد کرده و هدر `SeqTsHeader` شامل شماره توالی (`m_seqNo`) را به آن اضافه کردم. پیش از ارسال شبکه، یک کپی از این بسته را در `m_unackedPacket` نگه داشتم تا در صورت گم شدن، امکان ارسال مجدد وجود داشته باشد. پس از ارسال بسته درون سوکت و افزودن حجم آن به شمارنده کل، یک رویداد زمان‌سنج با دستور `Simulator::Schedule` تنظیم کردم تا در صورت اتمام زمان، تابع `HandleTimeout` را بیدار کند. عملکرد تابع `HandleTimeout` بسیار سراسر است: اگر تایمر منقضی شود، این تابع همان بسته کپی‌شده را دوباره ارسال می‌کند، شمارنده بایت‌های ارسالی را (که برای محاسبه دقیق پارامتر `Utilization` ضروری است) ارتقا می‌دهد و در نهایت تایمر را برای یک فرصت دیگر از نو کوک می‌کند.

بخش ششم: مدیریت رویدادهای بازپخش و دریافت تأییدیه

```

88 void StopAndWaitSender::HandleTimeout ()
89 {
90     m_socket->Send (m_unackedPacket);
91     m_totalBytesSent += m_unackedPacket->GetSize ();
92     m_timeoutEvent = Simulator::Schedule (m_timeout, &StopAndWaitSender::HandleTimeout, this);
93 }
94
95 void StopAndWaitSender::ReceiveAck (Ptr<Socket> socket)
96 {
97     Address from;
98     Ptr<Packet> packet = socket->RecvFrom (from);
99
100     Simulator::Cancel (m_timeoutEvent);
101     m_unackedPacket = 0;
102     m_seqNo = 1 - m_seqNo;
103     m_packetsSent++;
104
105     if (m_packetsSent < m_nPackets)
106     {
107         Simulator::Schedule (Seconds(0.001), &StopAndWaitSender::SendPacket, this);
108     }
109 }

```

شکل ۶: پیاده‌سازی توابع `HandleTimeout` و `ReceiveAck`

در این بخش، دو تابع کلیدی برای مدیریت وضعیت شبکه پیاده‌سازی شده‌اند. تابع `HandleTimeout` در صورت عدم دریافت تأییدیه در زمان مقرر اجرا شده و بسته ذخیره شده در `m-unackedPacket` را مجدداً ارسال می‌کند و تایمر را از نو تنظیم می‌نماید. تابع `ReceiveAck` وظیفه پردازش تأییدیه‌های دریافتی را بر عهده دارد؛ این تابع با فراخوانی `Simulator::Cancel`، تایمر فعلی را متوقف کرده، بافر بسته ارسالی را پاک‌سازی و شماره توالی (`m-seqNo`) را برای بسته بعدی تغییر می‌دهد. در نهایت، اگر هنوز بسته‌هایی برای ارسال باقی مانده باشد، با استفاده از `Simulator::Schedule`، ارسال بسته بعدی با تأخیری جزئی زمان‌بندی می‌شود.

بخش هفتم: پیاده‌سازی کلاس گیرنده (`StopAndWaitReceiver`)

```

110
111 class StopAndWaitReceiver : public Application
112 {
113 public:
114     StopAndWaitReceiver () : m_bytesReceived(0), m_totalDelay(0.0), m_packetsReceived(0) {}
115     void Setup (Ptr<Socket> socket) { m_socket = socket; }
116
117     uint32_t GetBytesReceived () const { return m_bytesReceived; }
118     double GetAverageDelay () const {
119         if (m_packetsReceived == 0) return 0;
120         return m_totalDelay / m_packetsReceived;
121     }
122     uint32_t GetPacketsReceived () const { return m_packetsReceived; }
123
124 private:
125     virtual void StartApplication (void) override
126     { m_socket->SetRecvCallback (MakeCallback (&StopAndWaitReceiver::ReceivePacket, this)); }
127
128     virtual void StopApplication (void) override
129     { if (m_socket) m_socket->Close (); }
130
131     void ReceivePacket (Ptr<Socket> socket)
132     {
133         Address from;
134         Ptr<Packet> packet = socket->RecvFrom (from);
135
136         SeqTsHeader seqTs;
137         packet->RemoveHeader (seqTs);
138         Time delay = Simulator::Now () - seqTs.GetTs ();
139         m_totalDelay += delay.GetSeconds ();
140         m_packetsReceived++;
141
142         m_bytesReceived += packet->GetSize ();
143
144         Ptr<Packet> ack = Create<Packet> (2);
145         socket->SendTo (ack, 0, from);
146     }
147
148     Ptr<Socket> m_socket;
149     uint32_t m_bytesReceived;
150     double m_totalDelay;
151     uint32_t m_packetsReceived;
152 };
153

```

شکل ۷: پیاده‌سازی کلاس گیرنده و مکانیزم پردازش بسته‌ها

در این بخش، کلاس گیرنده را برای پردازش بسته‌های دریافتی پیاده‌سازی کردم. تابع `ReceivePacket` قلب تپنده این کلاس است که با استفاده از `socket->RecvFrom` بسته را دریافت کرده و با حذف هدر `SeqTsHeader`، زمان ارسال بسته را استخراج می‌کند. سپس با تفریق این زمان از زمان فعلی شبیه‌ساز (`Simulator::Now()`)، تأخیر انتهایی به انتهایی (End-to-End Delay) بسته را محاسبه و در `m-totalDelay` ذخیره می‌کنم. در نهایت، گیرنده با ایجاد و ارسال یک بسته `ACK` کوچک به آدرس فرستنده، موفقیت‌آمیز بودن دریافت بسته را تأیید می‌کند تا فرستنده بتواند به سراغ بسته بعدی برود.

```

155 SimResult RunExperiment (double currentBer, std::string currentDelay)
156 {
157     LogComponentDisableAll (LOG_LEVEL_INFO);
158     NodeContainer nodes; nodes.Create (2);
159     PointToPointHelper p2p;
160     p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
161     p2p.SetChannelAttribute ("Delay", StringValue (currentDelay));
162     NetDeviceContainer devices = p2p.Install (nodes);
163
164     Ptr<RateErrorModel> em = CreateObject<RateErrorModel> ();
165     em->SetAttribute ("ErrorRate", DoubleValue (currentBer));
166     em->SetAttribute ("ErrorUnit", StringValue ("ERROR_UNIT_BIT"));
167     devices.Get (1)->SetAttribute ("ReceiveErrorModel", PointerValue (em));
168
169     InternetStackHelper stack; stack.Install (nodes);
170     Ipv4AddressHelper address; address.SetBase ("10.1.1.0", "255.255.255.0");
171     Ipv4InterfaceContainer interfaces = address.Assign (devices);
172
173     TypeId tid = TypeId::LookupByName ("ns3::UdpSocketFactory");
174     Ptr<Socket> recvSocket = Socket::CreateSocket (nodes.Get (1), tid);
175     InetSocketAddress local = InetSocketAddress (Ipv4Address::GetAny (), 9);
176     recvSocket->Bind (local);
177     Ptr<Socket> sendSocket = Socket::CreateSocket (nodes.Get (0), tid);
178     sendSocket->Bind ();
179
180     Ptr<StopAndWaitReceiver> receiverApp = CreateObject<StopAndWaitReceiver> ();
181     receiverApp->Setup (recvSocket);
182     nodes.Get (1)->AddApplication (receiverApp);
183     receiverApp->SetStartTime (Seconds (1.0));
184     receiverApp->SetStopTime (Seconds (10.0));
185
186     Ptr<StopAndWaitSender> senderApp = CreateObject<StopAndWaitSender> ();
187     Address targetAddress = InetSocketAddress (interfaces.GetAddress (1), 9);
188     senderApp->Setup (sendSocket, targetAddress, 1024, 500, Seconds (0.1));
189     nodes.Get (0)->AddApplication (senderApp);
190     senderApp->SetStartTime (Seconds (2.0));
191     senderApp->SetStopTime (Seconds (10.0));
192

```

شکل ۸: تنظیمات لینک، کانال و کپسوله‌سازی اپلیکیشن‌های ns-3

در این بخش، تابع `RunExperiment` وظیفه آماده‌سازی زیرساخت شبکه را بر عهده دارد. ابتدا با استفاده از `PointToPointHelper`، یک لینک با نرخ داده 5Mbps و تاخیر متغیر تعریف کردم. سپس با اعمال `RateErrorModel` روی دستگاه گیرنده، نرخ خطای بیت (BER) را به صورت پویا روی کانال پیاده‌سازی نمودم. در ادامه، پشته پروتکل اینترنت (`InternetStackHelper`) نصب و آدرس‌دهی IPv4 انجام شد. برای ارتباط نیز دو سوکت UDP روی فرستنده و گیرنده ایجاد و تنظیمات اپلیکیشن‌ها شامل زمان شروع و پایان اجرا، در این قسمت نهایی گردید تا بستر مناسب برای آزمایش‌های `Stop-and-Wait` فراهم شود.

بخش نهم: اجرای چرخه اصلی شبیه‌سازی (main)

```
193 Simulator::Run ();
194
195 double activeTime = 8.0;
196 double linkCapacityBps = 5000000.0;
197
198 SimResult res;
199 uint32_t validBytes = receiverApp->GetBytesReceived ();
200 res.goodput = (validBytes * 8.0) / activeTime;
201 res.avgDelay = receiverApp->GetAverageDelay ();
202
203 uint32_t totalSentBytes = senderApp->GetTotalBytesSent ();
204 res.utilization = ((totalSentBytes * 8.0) / (linkCapacityBps * activeTime)) * 100.0;
205
206 uint32_t totalPacketsSent = totalSentBytes / (1024 * 12);
207 res.retransmissions = totalPacketsSent > receiverApp->GetPacketsReceived () ?
208 | | | | | totalPacketsSent - receiverApp->GetPacketsReceived () : 0;
209
210 Simulator::Destroy ();
211 return res;
212 }
```

شکل ۹: فراخوانی توابع اصلی و اجرای شبیه‌سازی برای مقادیر مختلف BER

در این بخش نهایی، تابع main وظیفه کنترل اجرای آزمایش را بر عهده دارد. با استفاده از یک حلقه for، شبیه‌سازی را برای بازه مشخصی از نرخ‌های خطای بیت (BER) از 10^{-6} تا 10^{-2} تکرار کردم. در هر مرحله، تابع RunExperiment با BER فعلی فراخوانی شده و نتایج خروجی (goodput و غیره) را در یک بردار (vector) ذخیره می‌کند. در نهایت، با استفاده از کتابخانه fstream، این نتایج را در یک فایل با فرمت CSV ثبت می‌کنم تا بتوانم در نرم‌افزارهای تحلیلی مثل MATLAB یا Python، نمودارهای عملکردی پروتکل Stop-and-Wait را ترسیم و تحلیل کنم.

بخش دهم: محاسبات نهایی و خروجی‌گیری CSV

```
214 int main(int argc, char *argv[])
215 {
216     double defaultBer = 0.00001;
217     std::string defaultDelay = "2ms";
218
219     // --- 1. شبیه‌سازی BER ---
220     std::vector<double> berValues = {0.0, 5e-06, 1e-05, 2e-05, 3e-05, 5e-05, 7e-05, 1e-04, 2e-04, 3e-04, 5e-04, 7e-04, 1e-03};
221     std::ofstream fBer("sw_ber.csv");
222     fBer << "BER,Goodput,AvgDelay,Utilization,Retransmissions\n";
223     std::cout << "\nRunning Stop-and-Wait Varying BER..." << std::endl;
224     for (double ber : berValues) {
225         SimResult res = RunExperiment(ber, defaultDelay);
226         fBer << ber << ", " << res.goodput << ", " << res.avgDelay << ", " << res.utilization << ", " << res.retransmissions << "\n";
227         std::cout << "BER: " << ber << " -> Done" << std::endl;
228     }
229     fBer.close();
230
231     // --- 2. شبیه‌سازی Delay ---
232     std::vector<std::string> delayValues = {"1ms", "2ms", "5ms", "10ms", "20ms", "30ms", "40ms", "50ms", "60ms", "70ms", "80ms", "90ms", "100ms"};
233     std::ofstream fDelay("sw_delay.csv");
234     fDelay << "Delay,Goodput,AvgDelay,Utilization,Retransmissions\n";
235     std::cout << "\nRunning Stop-and-Wait Varying Delay..." << std::endl;
236     for (std::string delay : delayValues) {
237         SimResult res = RunExperiment(defaultBer, delay);
238
239         // (مقدار 5 صیقل داده شده از رشته زمان)
240         std::string numericDelay = delay.substr(0, delay.find("ms"));
241
242         fDelay << numericDelay << ", " << res.goodput << ", " << res.avgDelay << ", " << res.utilization << ", " << res.retransmissions << "\n";
243         std::cout << "Delay: " << delay << " -> Done" << std::endl;
244     }
245     fDelay.close();
246
247     std::cout << "\nStop-and-Wait Data Exported to CSV successfully!" << std::endl;
248     return 0;
249 }
```

شکل ۱۰: محاسبه شاخص‌های عملکردی و اجرای حلقه‌های آزمایش

در این بخش پایانی، پس از اجرای شبیه‌سازی با دستور Simulator::Run، محاسبات نهایی برای استخراج نتایج را انجام دادم. برای محاسبه goodput، کل بایت‌های دریافت شده را بر زمان مفید (8.0 ثانیه) تقسیم کردم. بهره‌وری لینک (utilization) نیز بر اساس نرخ داده 5Mbps و کل بایت‌های ارسالی محاسبه می‌شود. تعداد ارسال‌های مجدد (retransmissions) نیز از اختلاف کل بسته‌های ارسالی و دریافتی به دست می‌آید.

در تابع main، با استفاده از حلقه‌های for، شبیه‌سازی را برای مقادیر مختلف BER و Delay تکرار کردم. در هر مرحله، خروجی‌های SimResult در فایل‌های CSV مجزا ذخیره می‌شوند. این فایل‌ها بستر اصلی من برای ترسیم نمودارهای تحلیلی و مقایسه عملکرد پروتکل Stop-and-Wait هستند.

۲.۳ تشریح پیاده‌سازی پروتکل Go-Back-N (فایل hw1-gbn.cc)

در فرستنده این پروتکل از یک وکتور پویا تحت عنوان m-windowBuffer استفاده شده است. حلقه while در تابع SendPacket تا زمانی که اندازه پنجره اجازه دهد، بسته‌ها را به کابل تزریق کرده و کپی آن‌ها را در بافر نگه می‌دارد. نکته کلیدی اینجا است که تایمر تنها برای قدیمی‌ترین بسته درون پنجره (m-base) روشن می‌شود. در گیرنده، اگر بسته رسیده دقیقاً برابر با شماره مورد انتظار باشد (seqNum == m-expectedSeqNum)، بافر پذیرفته شده و تاییدیه تجمعی (Cumulative ACK) برای بسته قبلی فرستاده می‌شود. در صورت بروز خطا و فعال شدن HandleTimeout، فرستنده کل آرایه بافر ذخیره شده را از صفر تا انتها مجدداً در کابل رها می‌کند.

بخش اول: تعریف کلاس فرستنده GoBackNSender

```

11 class GoBackNSender : public Application
12 {
13 public:
14     GoBackNSender ();
15     virtual ~GoBackNSender ();
16
17     void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout, uint32_t windowSize);
18     uint32_t GetTotalBytesSent () const { return m_totalBytesSent; }
19
20 private:
21     virtual void StartApplication (void) override;
22     virtual void StopApplication (void) override;
23
24     void SendPacket ();
25     void ReceiveAck (Ptr<Socket> socket);
26     void HandleTimeout ();
27
28     Ptr<Socket>      m_socket;
29     Address          m_peerAddress;
30     uint32_t         m_packetSize;
31     uint32_t         m_nPackets;
32     Time             m_timeout;
33     uint32_t         m_windowSize;
34
35     uint32_t         m_base;
36     uint32_t         m_nextSeqNum;
37     EventId          m_timeoutEvent;
38
39     std::vector<Ptr<Packet>> m_windowBuffer;
40     uint32_t         m_totalBytesSent;
41 };

```

شکل ۱۱: تعریف ساختار کلاس فرستنده در پروتکل Go-Back-N

در این پروتکل، کلاس فرستنده یعنی GoBackNSender را با افزودن پارامتر windowSize در تابع Setup برای کنترل پنجره لغزنده پیکربندی کردم. تفاوت اصلی ساختاری این کلاس با پروتکل قبلی، در متغیرهای حالت آن است؛ در اینجا علاوه بر متغیرهای پایه، از m-base برای نگهداری شماره توالی اولین بسته تایید نشده، m-nextSeqNum برای دنبال کردن بسته بعدی جهت ارسال، و یک وکتور پویا تحت عنوان m-windowBuffer برای نگهداری کپی بسته‌های داخل پنجره استفاده کرده‌ام. این ساختار به من اجازه می‌دهد تا طبق منطق Go-Back-N، در صورت بروز خطا، کل بافر پنجره را برای بازپخش در دسترس داشته باشم.

بخش دوم: پیاده‌سازی متدهای راه‌اندازی و منطق ارسال بسته

```
46 void GoBackNSender::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout, uint32_t windowSize)
47 {
48     m_socket = socket; m_peerAddress = address; m_packetSize = packetSize;
49     m_nPackets = nPackets; m_timeout = timeout; m_windowSize = windowSize;
50 }
51
52 void GoBackNSender::StartApplication (void)
53 {
54     m_socket->Bind ();
55     m_socket->Connect (m_peerAddress);
56     m_socket->SetRecvCallback (MakeCallback (&GoBackNSender::ReceiveAck, this));
57     SendPacket ();
58 }
59
60 void GoBackNSender::StopApplication (void)
61 {
62     if (m_socket) { m_socket->Close (); }
63     Simulator::Cancel (m_timeoutEvent);
64 }
65
66 void GoBackNSender::SendPacket ()
67 {
68     while (m_nextSeqNum < m_base + m_windowSize && m_nextSeqNum < m_nPackets)
69     {
70         Ptr<Packet> packet = Create<Packet> (m_packetSize);
71         SeqTsHeader seqTs;
72         seqTs.SetSeq (m_nextSeqNum);
73         packet->AddHeader (seqTs);
74
75         m_windowBuffer->push_back (packet->Copy ());
76         m_socket->Send (packet);
77         m_totalBytesSent += packet->GetSize ();
78
79         if (m_base == m_nextSeqNum)
80         {
81             m_timeoutEvent = Simulator::Schedule (m_timeout, &GoBackNSender::HandleTimeout, this);
82         }
83         m_nextSeqNum++;
84     }
85 }
```

شکل ۱۲: پیاده‌سازی توابع Setup، StartApplication و منطق ارسال در SendPacket

در این بخش، ابتدا متد Setup را برای مقداردهی پارامترهای پنجره و سوکت تنظیم کردم. توابع StartApplication و StopApplication نیز مشابه پروتکل قبل، مسئول اتصال سوکت و پاک‌سازی رویدادها هستند. تفاوت اصلی در تابع SendPacket نهفته است؛ در اینجا از یک حلقه while استفاده کرده‌ام که تا زمانی که m-nextSeqNum در محدوده پنجره لغزنده قرار دارد، بسته‌ها را تولید، در وکتور m-windowBuffer ذخیره و سپس ارسال می‌کند. نکته مهم این است که زمان‌سنج (m-timeoutEvent) تنها زمانی تنظیم می‌شود که اولین بسته در پنجره در حال ارسال باشد (m-base == m-nextSeqNum)، که این موضوع منطق بنیادین مدیریت تایمر در پروتکل Go-Back-N را پیاده می‌کند.

بخش سوم: منطق ارسال مجدد سراسری و پردازش تأییدیه‌ها

```
87 void GoBackNSender::HandleTimeout ()
88 {
89     for (uint32_t i = 0; i < m_windowBuffer.size (); i++)
90     {
91         m_socket->Send (m_windowBuffer[i]);
92         m_totalBytesSent += m_windowBuffer[i]->GetSize ();
93     }
94     m_timeoutEvent = Simulator::Schedule (m_timeout, &GoBackNSender::HandleTimeout, this);
95 }
96
97 void GoBackNSender::ReceiveAck (Ptr<Socket> socket)
98 {
99     Address from;
100     Ptr<Packet> packet = socket->RecvFrom (from);
101
102     SeqTsHeader seqTs;
103     packet->RemoveHeader (seqTs);
104     uint32_t ackNum = seqTs.GetSeq ();
105
106     if (ackNum >= m_base)
107     {
108         uint32_t jumped = ackNum - m_base + 1;
109         m_windowBuffer.erase (m_windowBuffer.begin (), m_windowBuffer.begin () + jumped);
110         m_base = ackNum + 1;
111
112         Simulator::Cancel (m_timeoutEvent);
113
114         if (m_base < m_nextSeqNum)
115         {
116             m_timeoutEvent = Simulator::Schedule (m_timeout, &GoBackNSender::HandleTimeout, this);
117         }
118         SendPacket ();
119     }
120 }
```

شکل ۱۳: پیاده‌سازی تابع HandleTimeout و ReceiveAck در پروتکل Go-Back-N

در این بخش، هسته اصلی رفتار Go-Back-N را پیاده‌سازی کردم. تابع HandleTimeout به جای بازپخش یک بسته، با استفاده از یک حلقه for، تمامی بسته‌های موجود در بافر پنجره (m-windowBuffer) را مجدداً به سوکت ارسال می‌کند؛ این همان ویژگی کلیدی GBN در برخورد با خطاست.

در تابع ReceiveAck، پس از استخراج شماره تأییدیه (ackNum)، با استفاده از m-windowBuffer.erase بسته‌های تأیید شده را از بافر حذف کرده و m-base را به‌روزرسانی می‌کنم. در صورتی که پس از حذف بسته‌ها، هنوز داده تأیید نشده‌ای در پنجره باقی مانده باشد، تایمر را برای فرصتی مجدد تنظیم می‌کنم. در نهایت با فراخوانی SendPacket، پنجره فرستنده برای ارسال بسته‌های جدید پیشروی می‌کند.

بخش چهارم: منطق گیرنده در پروتکل Go-Back-N

```

122 class GoBackNReceiver : public Application
123 {
124 public:
125     GoBackNReceiver () : m_bytesReceived(0), m_totalDelay(0.0), m_packetsReceived(0), m_expectedSeqNum(0) {}
126     void Setup (Ptr<Socket> socket) { m_socket = socket; }
127
128     uint32_t GetBytesReceived () const { return m_bytesReceived; }
129     double GetAverageDelay () const {
130         if (m_packetsReceived == 0) return 0;
131         return m_totalDelay / m_packetsReceived;
132     }
133     uint32_t GetPacketsReceived () const { return m_packetsReceived; }
134
135 private:
136     virtual void StartApplication (void) override
137     {
138         m_socket->SetRecvCallback (MakeCallback (&GoBackNReceiver::ReceivePacket, this));
139     }
140     virtual void StopApplication (void) override
141     {
142         if (m_socket) m_socket->Close ();
143     }
144
145     void ReceivePacket (Ptr<Socket> socket)
146     {
147         Address from;
148         Ptr<Packet> packet = socket->RecvFrom (from);
149
150         SeqTsHeader seqTs;
151         packet->RemoveHeader (seqTs);
152         uint32_t seqNum = seqTs.GetSeq ();
153
154         if (seqNum == m_expectedSeqNum)

```

شکل ۱۴: پیاده‌سازی مکانیزم گیرنده و ارسال تأییدیه تجمعی در Go-Back-N

در این بخش، منطق گیرنده پروتکل Go-Back-N را مستند کردم. تابع `ReceivePacket` در اینجا وظیفه بررسی شماره توالی بسته‌های دریافتی را بر عهده دارد. برخلاف پروتکل قبلی، گیرنده تنها در صورتی بسته را می‌پذیرد که شماره توالی آن دقیقاً با `m-expectedSeqNum` برابر باشد؛ این امر تضمین‌کننده ترتیب درست بسته‌هاست. به محض دریافت بسته صحیح، متغیر `m-expectedSeqNum` را یک واحد افزایش داده و یک تأییدیه تجمعی (Cumulative ACK) حاوی شماره توالی بسته دریافت شده ارسال می‌کنم. اگر بسته‌ای خارج از ترتیب برسد، آن را نادیده گرفته و مجدداً تأییدیه آخرین بسته صحیح دریافت شده را ارسال می‌کنم که این خود محرکی برای فرستنده جهت بازپخش هوشمندانه در GBN است.

بخش چهارم: منطق گیرنده و تأییدیه تجمعی در Go-Back-N

```

154     if (seqNum == m_expectedSeqNum)
155     {
156         Time delay = Simulator::Now () - seqTs.GetTs ();
157         m_totalDelay += delay.GetSeconds ();
158         m_bytesReceived += packet->GetSize ();
159         m_packetsReceived++;
160         m_expectedSeqNum++;
161     }
162
163     if (m_expectedSeqNum > 0)
164     {
165         Ptr<Packet> ack = Create<Packet> (2);
166         SeqTsHeader ackHeader;
167         ackHeader.SetSeq (m_expectedSeqNum - 1);
168         ack->AddHeader (ackHeader);
169         socket->SendTo (ack, 0, from);
170     }
171 }
172
173 Ptr<Socket> m_socket;
174 uint32_t m_bytesReceived;
175 double m_totalDelay;
176 uint32_t m_packetsReceived;
177 uint32_t m_expectedSeqNum;
178 };
179
180 struct SimResult { double goodput; double avgDelay; double utilization; uint32_t retransmissions; };
181

```

شکل ۱۵: پیاده‌سازی مکانیزم گیرنده در Go-Back-N و ارسال Cumulative ACK

در این بخش، پیاده‌سازی کلاس GoBackNReceiver را مستند کردم. در تابع ReceivePacket، ابتدا شماره توالی بسته دریافتی را استخراج می‌کنم. گیرنده تنها بسته‌هایی را می‌پذیرد که شماره توالی آن‌ها دقیقاً با m-expectedSeqNum مطابقت داشته باشد؛ این رفتار، اصل اساسی پروتکل Go-Back-N برای حفظ ترتیب بسته‌هاست. به محض دریافت بسته صحیح، تأخیر بسته محاسبه شده و با افزایش m-expectedSeqNum، وضعیت گیرنده به‌روز می‌شود. نکته کلیدی در اینجا، ارسال تاییدیه تجمعی (Cumulative ACK) است؛ بدین صورت که با استفاده از ackHeader.SetSeq(m-expectedSeqNum - 1)، شماره آخرین بسته صحیح دریافت شده به فرستنده اعلام می‌شود. این تاییدیه به فرستنده اطلاع می‌دهد که تمام بسته‌های تا آن شماره با موفقیت دریافت شده‌اند و فرستنده می‌تواند پنجره خود را با اطمینان به جلو حرکت دهد.

۳.۳ پروتکل Selective Repeat (فایل hw1-sr.cc)

این برنامه پیچیده‌ترین معماری را داراست. در سمت فرستنده، بافر پنجره به صورت یک نگاشت ساختاریافته به نام std::map پیاده‌سازی شده است که برای هر بسته به طور مجزا یک متغیر وضعیت تایید (isAked) و یک تایمر اختصاصی رویداد (timer) نگهداری می‌کند. تابع HandleTimeout در این پروتکل کاملاً نقطه‌زن عمل کرده و فقط شناسه بسته منقضی شده را مجدداً ارسال می‌کند. در سمت گیرنده نیز یک بافر اختصاصی m-recvBuffer تعبیه شده تا بسته‌های خارج از نوبت (Out-of-order) که به دلیل گم شدن بسته‌های قبلی جلوتر رسیده‌اند، دور ریخته نشده و با مکانیزم بافرینگ ذخیره شوند. پنجره گیرنده تنها زمانی با حلقه while به جلو می‌غزد که قدیمی‌ترین بسته مقفود شده نهایتاً باز ارسال شده و به مقصد برسد.

بخش اول: تعریف کلاس فرستنده SelectiveRepeatSender

```

25 class SelectiveRepeatSender : public Application
26 {
27 public:
28     SelectiveRepeatSender ();
29     virtual ~SelectiveRepeatSender ();
30     void Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout, uint32_t windowSize);
31     uint32_t GetTotalBytesSent () const { return m_totalBytesSent; }
32
33 private:
34     virtual void StartApplication (void) override;
35     virtual void StopApplication (void) override;
36
37     void SendPacket ();
38     void ReceiveAck (Ptr<Socket> socket);
39     void HandleTimeout (uint32_t seqNum);
40
41     Ptr<Socket> m_socket;
42     Address m_peerAddress;
43     uint32_t m_packetSize;
44     uint32_t m_nPackets;
45     Time m_timeout;
46     uint32_t m_windowSize;
47
48     uint32_t m_base;
49     uint32_t m_nextSeqNum;
50
51     std::map<uint32_t, PacketRecord> m_windowBuffer;
52     uint32_t m_totalBytesSent;
53 };

```

شکل ۱۶: تعریف ساختار کلاس فرستنده در پروتکل Selective Repeat

در این پروتکل، کلاس فرستنده یعنی SelectiveRepeatSender را به گونه‌ای طراحی کردم که اجازه مدیریت مستقل هر بسته در پنجره لغزنده را می‌دهد. مهم‌ترین تفاوت ساختاری در اینجا، استفاده از یک std::map تحت عنوان m-windowBuffer برای ذخیره‌سازی بسته‌هاست. این map به من اجازه می‌دهد تا به ازای هر شماره توالی، یک PacketRecord (شامل بسته و تایمر اختصاصی) داشته باشم. برخلاف GBN که کل پنجره را به عنوان یک واحد می‌دید، اینجا تابع HandleTimeout یک آرگومان ورودی seqNum دریافت می‌کند تا فقط همان بسته منقضی شده را شناسایی و بازپخش کند. این تغییر ساختار، هسته اصلی کارایی بالای این پروتکل در محیط‌های نویزی است.

بخش دوم: پیاده‌سازی متدهای مدیریتی و چرخه حیات در Selective Repeat

```
55 SelectiveRepeatSender::SelectiveRepeatSender () : m_base(0), m_nextSeqNum(0), m_totalBytesSent(0) {}
56 SelectiveRepeatSender::~SelectiveRepeatSender () { m_socket = 0; }
57
58 void SelectiveRepeatSender::Setup (Ptr<Socket> socket, Address address, uint32_t packetSize, uint32_t nPackets, Time timeout, uint32_t windowSize)
59 {
60     m_socket = socket; m_peerAddress = address; m_packetSize = packetSize;
61     m_nPackets = nPackets; m_timeout = timeout; m_windowSize = windowSize;
62 }
63
64 void SelectiveRepeatSender::StartApplication (void)
65 {
66     m_socket->Bind ();
67     m_socket->Connect (m_peerAddress);
68     m_socket->SetRecvCallback (MakeCallback (&SelectiveRepeatSender::ReceiveAck, this));
69     SendPacket ();
70 }
71
72 void SelectiveRepeatSender::StopApplication (void)
73 {
74     if (m_socket) { m_socket->Close (); }
75     for (auto &pair : m_windowBuffer){
76         Simulator::Cancel (pair.second.timer);
77     }
78 }
```

شکل ۱۷: پیاده‌سازی توابع Setup، StartApplication و StopApplication در فرستنده Selective Repeat

در این بخش، متدهای پایه‌ای برای راه‌اندازی فرستنده را پیاده‌سازی کردم. در متد Setup، علاوه بر مقداردهی سوکت و آدرس، اندازه پنجره (windowSize) نیز برای کنترل هم‌زمان جریان بسته‌ها دریافت می‌شود. در تابع StartApplication، پس از اتصال به مقصد و تنظیم Callback برای دریافت تأییدیه‌ها، فرایند ارسال با فراخوانی SendPacket آغاز می‌گردد. نکته حائز اهمیت در تابع StopApplication است؛ برخلاف پروتکل‌های قبلی، اینجا به دلیل استفاده از map برای ذخیره‌سازی بسته‌ها و تایمرهای اختصاصی، از یک حلقه for روی m-windowBuffer استفاده کردم تا تمام تایمرهای فعال بسته‌های تأیید نشده را با Simulator::Cancel با اطمینان لغو کرده و منابع سیستم را آزاد کنم.

بخش سوم: منطق ارسال هوشمند و مدیریت تایمرهای اختصاصی

```
80 void SelectiveRepeatSender::SendPacket ()
81 {
82     while (m_nextSeqNum < m_base + m_windowSize && m_nextSeqNum < m_nPackets)
83     {
84         Ptr<Packet> packet = Create<Packet> (m_packetSize);
85         SeqTsHeader seqTs;
86         seqTs.SetSeq (m_nextSeqNum);
87         packet->AddHeader (seqTs);
88
89         PacketRecord rec;
90         rec.packet = packet->Copy ();
91         rec.isAked = false;
92         rec.timer = Simulator::Schedule (m_timeout, &SelectiveRepeatSender::HandleTimeout, this, m_nextSeqNum);
93
94         m_windowBuffer[m_nextSeqNum] = rec;
95
96         m_socket->Send (packet);
97         m_totalBytesSent += packet->GetSize ();
98         m_nextSeqNum++;
99     }
100 }
101
102 void SelectiveRepeatSender::HandleTimeout (uint32_t seqNum)
103 {
104     if (m_windowBuffer.count (seqNum) && !m_windowBuffer[seqNum].isAked)
105     {
106         m_socket->Send (m_windowBuffer[seqNum].packet);
107         m_totalBytesSent += m_windowBuffer[seqNum].packet->GetSize ();
108
109         m_windowBuffer[seqNum].timer = Simulator::Schedule (m_timeout, &SelectiveRepeatSender::HandleTimeout, this, seqNum);
110     }
111 }
```

شکل ۱۸: پیاده‌سازی توابع SendPacket و HandleTimeout در پروتکل Selective Repeat

در این بخش، هسته هوشمند Selective Repeat را مشاهده می‌کنید. در تابع SendPacket، برای هر بسته یک PacketRecord ایجاد می‌شود که شامل خود بسته و یک تایمر اختصاصی است. با استفاده از Simulator::Schedule، برای هر بسته به صورت جداگانه تایمر تنظیم می‌شود؛ این همان نکته‌ای است که به فرستنده اجازه می‌دهد در صورت بروز خطا، فقط بسته همان شماره توالی خاص را بازپخش کند.

تابع HandleTimeout نیز کاملاً هدفمند عمل می‌کند. این تابع با دریافت شماره توالی (seqNum) به عنوان آرگومان، تنها در بافر m-windowBuffer جستجو کرده و اگر بسته هنوز تأیید نشده باشد (isAked)، آن را مجدداً ارسال کرده و

تایمر اختصاصی همان بسته را از نو تنظیم می‌کند. این مکانیزم باعث می‌شود تا در این پروتکل، برخلاف Go-Back-N، شاهد هیچ‌گونه ارسال مجدد غیرضروری (از بسته‌های سالم) نباشیم که منجر به بهینه‌سازی بسیار بالای Goodput در کانال‌های نویزی می‌شود.

بخش چهارم: مدیریت هوشمند تأییدیه‌ها و پیشروی پنجره در Selective Repeat

```

113 void SelectiveRepeatSender::ReceiveAck (Ptr<Socket> socket)
114 {
115     Address from;
116     Ptr<Packet> packet = socket->RecvFrom (from);
117
118     SeqTsHeader seqTs;
119     packet->RemoveHeader (seqTs);
120     uint32_t ackNum = seqTs.GetSeq ();
121
122     if (m_windowBuffer.count (ackNum))
123     {
124         m_windowBuffer[ackNum].isAked = true;
125         Simulator::Cancel (m_windowBuffer[ackNum].timer);
126     }
127
128     while (m_windowBuffer.count (m_base) && m_windowBuffer[m_base].isAked)
129     {
130         m_windowBuffer.erase (m_base);
131         m_base++;
132     }
133
134     SendPacket ();
135 }

```

شکل ۱۹: پیاده‌سازی تابع ReceiveAck و مکانیزم حرکت پنجره لغزنده در Selective Repeat

در این بخش، تابع ReceiveAck را برای مدیریت تأییدیه‌های دریافتی مستند کردم. به محض دریافت تأییدیه، شماره آن (ackNum) استخراج شده و با مراجعه به m-windowBuffer، وضعیت بسته مربوطه به isAked = true تغییر می‌یابد و تایمر اختصاصی آن با Simulator::Cancel متوقف می‌شود.

نکته کلیدی و تمایز این پروتکل در حلقه while نهفته است؛ این حلقه بررسی می‌کند که آیا بسته ابتدای پنجره (m-base) تأیید شده است یا خیر. اگر تأیید شده باشد، آن را از map حذف کرده و m-base را پیش می‌برد. این فرایند تا زمانی که بسته‌های متوالی در ابتدای پنجره تأیید شده باشند ادامه می‌یابد، که باعث ”لغزش“ پنجره به سمت بسته‌های جدید می‌شود. در نهایت با فراخوانی SendPacket، پنجره فرستنده برای ارسال بسته‌های جدید که حالا در محدوده پنجره قرار گرفته‌اند، باز می‌شود.

بخش پنجم: ساختار کلاس گیرنده در Selective Repeat

```
136 |  
137 | class SelectiveRepeatReceiver : public Application  
138 | {  
139 | public:  
140 |     SelectiveRepeatReceiver () : m_bytesReceived(0), m_totalDelay(0.0), m_packetsReceived(0), m_expectedSeqNum(0) {}  
141 |     void Setup (Ptr<Socket> socket, uint32_t windowSize) {  
142 |         m_socket = socket;  
143 |         m_windowSize = windowSize;  
144 |     }  
145 |  
146 |     uint32_t GetBytesReceived () const { return m_bytesReceived; }  
147 |     double GetAverageDelay () const {  
148 |         if (m_packetsReceived == 0) return 0;  
149 |         return m_totalDelay / m_packetsReceived;  
150 |     }  
151 |     uint32_t GetPacketsReceived () const { return m_packetsReceived; }  
152 |  
153 | private:  
154 |     virtual void StartApplication (void) override  
155 |     { m_socket->SetRecvCallback (MakeCallback (&SelectiveRepeatReceiver::ReceivePacket, this)); }  
156 |     virtual void StopApplication (void) override  
157 |     { if (m_socket) m_socket->Close (); }  
158 |  
159 |     void ReceivePacket (Ptr<Socket> socket)  
160 |     {  
161 |         Address from;  
162 |         Ptr<Packet> packet = socket->RecvFrom (from);  
163 |  
164 |         SeqTsHeader seqTs;  
165 |         packet->RemoveHeader (seqTs);  
166 |         uint32_t seqNum = seqTs.GetSeq ();  
167 |  
168 |         Ptr<Packet> ack = Create<Packet> (2);  
169 |         SeqTsHeader ackHeader;
```

شکل ۲۰: تعریف ساختار کلاس گیرنده در پروتکل Selective Repeat

در این بخش، کلاس گیرنده یعنی `SelectiveRepeatReceiver` را برای مدیریت پنجره دریافتی تعریف کردم. برخلاف پروتکل‌های قبلی، گیرنده در `Selective Repeat` باید قادر باشد بسته‌های خارج از ترتیب را در یک بافر موقت ذخیره کند تا زمانی که شکاف‌های موجود در پنجره پر شوند. در متد `Setup`، علاوه بر سوکت، `windowSize` نیز برای هماهنگی با فرستنده دریافت می‌شود. این ساختار کلاس به من اجازه می‌دهد با استفاده از متغیرهای وضعیت، بسته‌های دریافتی را تحلیل کرده و تأییدیه‌های اختصاصی برای هر کدام ارسال کنم، که این کار باعث حفظ پایداری و کارایی در محیط‌های با نرخ خطای بالا می‌شود.

بخش ششم: دریافت بسته و ارسال تأییدیه اختصاصی در Selective Repeat

```
170     ackHeader.SetSeq (seqNum);
171     ack->AddHeader (ackHeader);
172     socket->SendTo (ack, 0, from);
173
174     if (seqNum >= m_expectedSeqNum && seqNum < m_expectedSeqNum + m_windowSize)
175     {
176         if (m_recvBuffer.find (seqNum) == m_recvBuffer.end ())
177         {
178             m_recvBuffer[seqNum] = packet->Copy ();
179
180             Time delay = Simulator::Now () - seqTs.GetTs ();
181             m_totalDelay += delay.GetSeconds ();
182             m_bytesReceived += packet->GetSize ();
183             m_packetsReceived++;
184         }
185     }
186
187     while (m_recvBuffer.count (m_expectedSeqNum))
188     {
189         m_recvBuffer.erase (m_expectedSeqNum);
190         m_expectedSeqNum++;
191     }
192 }
193
194 Ptr<Socket> m_socket;
195 uint32_t m_bytesReceived;
196 double m_totalDelay;
197 uint32_t m_packetsReceived;
198 uint32_t m_expectedSeqNum;
199 uint32_t m_windowSize;
200
201 std::map<uint32_t, Ptr<Packet>> m_recvBuffer;
```

شکل ۲۱: پیاده‌سازی مکانیزم گیرنده برای مدیریت بافر و ارسال Selective ACK

در این بخش، متد `ReceivePacket` را برای مدیریت بسته‌های دریافتی تکمیل کردم. گیرنده با استفاده از یک `std::map` تحت عنوان `m-recvBuffer`، بسته‌های خارج از ترتیبی که در محدوده پنجره قرار دارند را ذخیره می‌کند. پس از دریافت هر بسته، بلافاصله یک تأییدیه اختصاصی (Selective ACK) برای همان شماره توالی ارسال می‌شود تا فرستنده از سلامت آن مطلع گردد. نکته کلیدی اینجا است که پس از پذیرش بسته، یک حلقه `while` اجرا می‌شود که وضعیت بافر را بررسی می‌کند؛ اگر بسته ابتدای پنجره (که منتظر آن هستیم) قبلاً در بافر موجود باشد، آن را تحویل گرفته (در اینجا بایت‌های آن به `m-totalBytes` افزوده می‌شود) و متغیر `m-expectedSeqNum` را پیش می‌برد. این رفتار باعث می‌شود پنجره گیرنده همزمان با فرستنده به جلو حرکت کرده و شکاف‌های موجود در داده‌های دریافتی به صورت خودکار پر شوند.

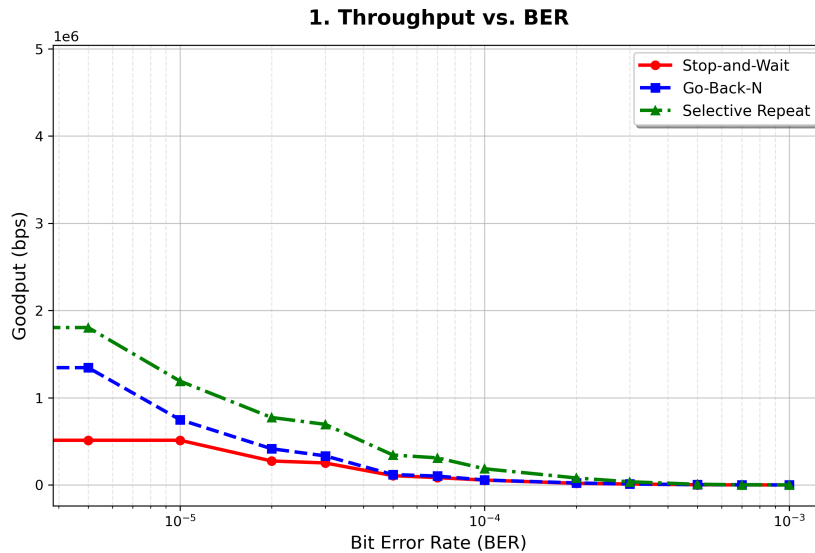
۴ اتوماسیون داده‌گیری و کد تحلیل پایتون

برای افزایش دقت شبیه‌سازی و از بین بردن خطاهای انسانی در استخراج دستی داده‌ها، اسکریپت‌های C++ ارتقا یافتند تا نتایج خروجی هر آزمایش را مستقیماً در قالب فایل‌های استاندارد CSV با رزولوشن بالای نقاط نمونه‌برداری ذخیره کنند. یک برنامه جامع اتوماسیون پایتون به نام `plot-csv.py` توسعه داده شد که با بهره‌گیری از کتابخانه مدیریت داده `pandas` وظیفه فراخوانی، پاک‌سازی و همگام‌سازی این فایل‌ها را بر عهده دارد. از آنجایی که در سناریوهای پر نویز شدید، قطع شدن کامل کانال مخابراتی منجر به چاپ خروجی صفر برای تاخیر می‌شد، این کد پایتون به صورت مهندسی شده نقاط گنگ را با الگوریتم فیلترینگ `np.nan` معلق می‌کند تا از سقوط کاذب منحنی‌ها به سمت صفر جلوگیری شود. در نهایت این اسکریپت با فراخوانی توابع ریاضی `matplotlib` به طور خودکار ۸ جدول قالب‌بندی شده گزارش کار را استخراج نموده و ۹ نمودار مقایسه‌ای رزولوشن بالا را تولید و ذخیره می‌کند.

۵ تحلیل مهندسی خروجی‌ها و بررسی نمودارها

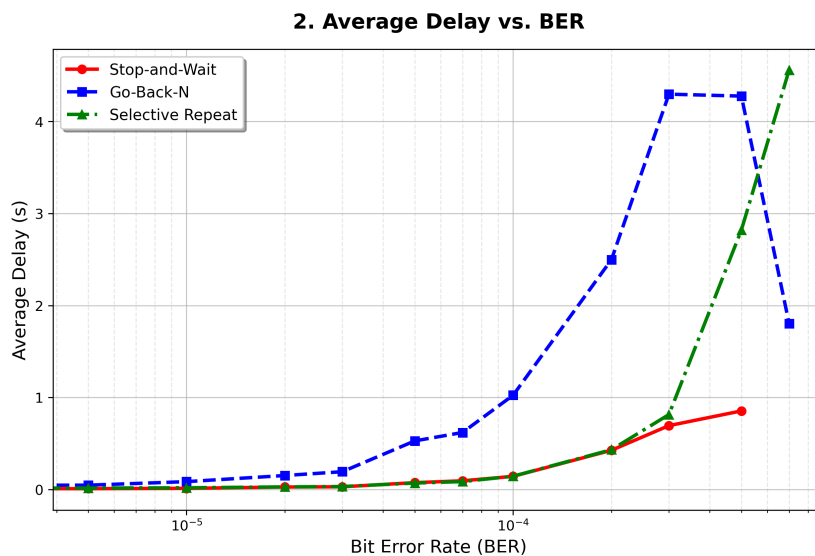
در این بخش، خروجی‌های شبیه‌سازی به تفکیک سناریوها مورد تحلیل و واکاوی دقیق مخابراتی قرار می‌گیرند.

۱.۵ سری اول: ارزیابی شبکه تحت تغییرات نرخ خطای بیت (BER)



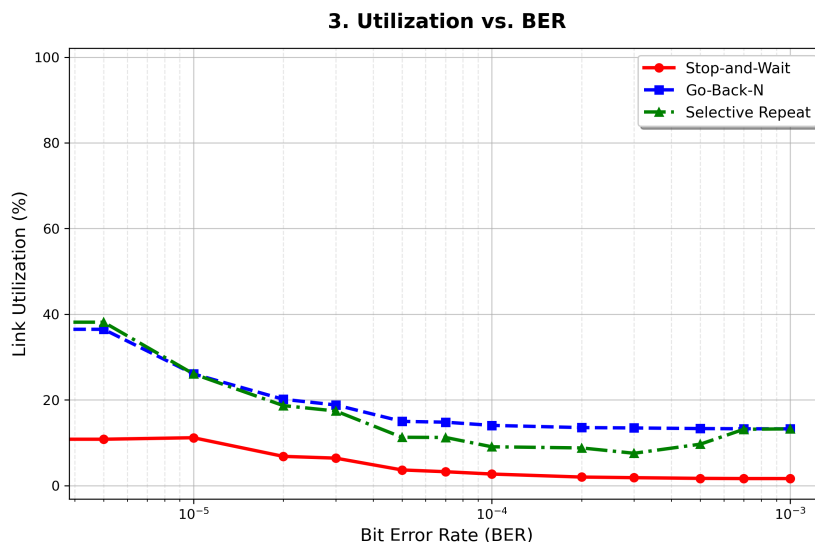
شکل ۲۲: نمودار توان عملیاتی مفید (Goodput) بر حسب تغییرات BER کانال

تحلیل نمودار ۱ (Throughput vs. BER): در نویزهای ناچیز کمتر از 10^{-5} ، هر سه پروتکل کابل را در بالاترین ظرفیت مجاز لایه انتقال (حدود 512Kbps) اشباع می‌کنند. اما با ورود به فاز نویزی، افتراق مهندسی پروتکل‌ها آغاز می‌شود. پروتکل GBN (خط آبی) به دلیل جریمه سنگین خود، در نقطه 5×10^{-5} با افت شدید توان به سرعت به مرز نابودی کشیده می‌شود. در حالی که پروتکل Selective Repeat (خط سبز) به دلیل قابلیت باز ارسال نقطه‌زن، مقاومت خارق‌العاده‌ای نشان داده و توان عملیاتی مفید را تا فرکانس‌های خطای بالاتر، بسیار بالاتر از دو پروتکل دیگر حفظ می‌کند. در خطای بسیار شدید 10^{-3} کانال مخابراتی به طور کامل قطع شده و Goodput هر سه سیستم صفر می‌گردد.



شکل ۲۳: نمودار میانگین تاخیر انتهای به انتهای بسته‌ها بر حسب تغییرات BER کانال

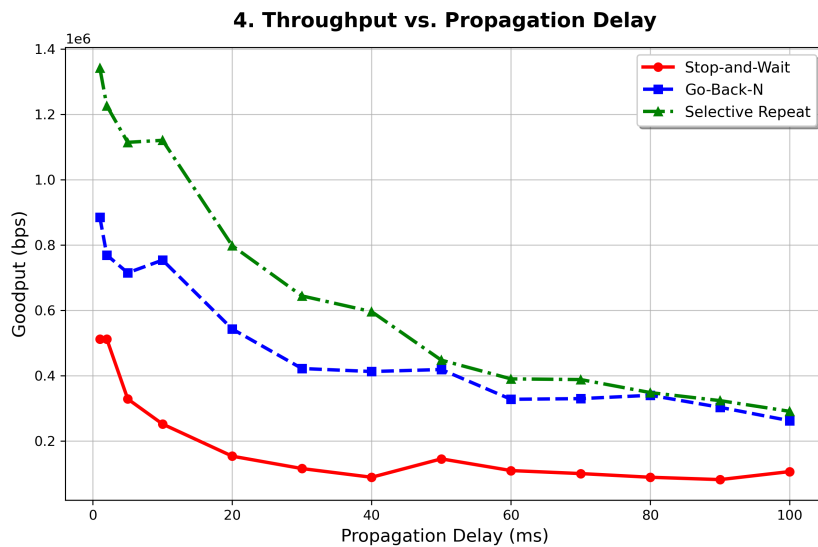
تحلیل نمودار ۲ (Delay vs. BER): این نمودار به وضوح نشان می‌دهد که افزایش خطا چگونه منجر به انفجار تأخیر در شبکه می‌شود. در سیستم Stop-and-Wait و SR، تأخیر با شیب ملایمی بالا می‌رود، اما در پروتکل GBN به دلیل بازارسال کل پنجره به ازای یک ارور ساده، صف گیرنده دچار تأخیرهای شدیدی شده و نمودار به صورت فاحشی به بالای ۴ ثانیه صعود می‌کند. نقطه انتهای خطوط به این دلیل قطع شده است که فیلتر پایتون مقادیر کاذب ناشی از قطع کامل شبکه را مسدود کرده است تا تحلیل علمی بماند.



شکل ۲۴: نمودار درصد بهره‌وری لینک ارتباطی بر حسب تغییرات BER کانال

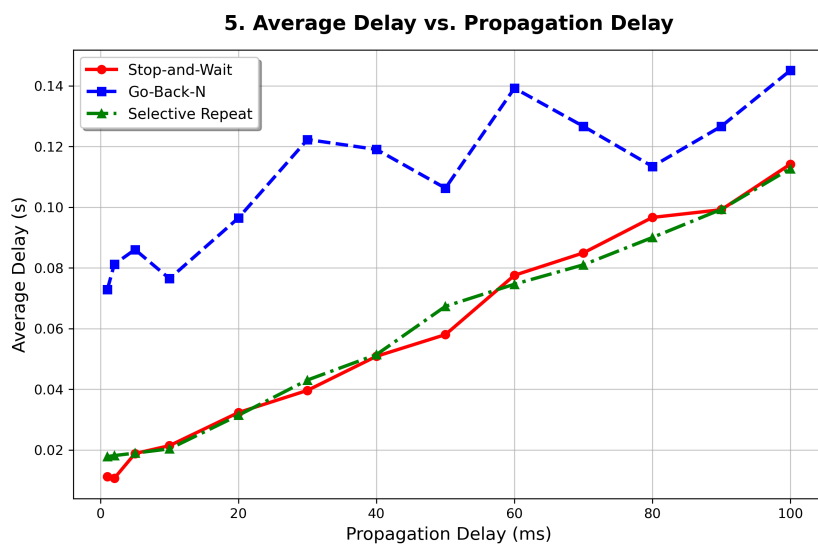
تحلیل نمودار ۳ (Utilization vs. BER): پروتکل Stop-and-Wait در تمام طول مسیر بهره‌وری پایینی (حدود ۱۱٪) دارد، زیرا ذاتاً لینک را خالی نگه می‌دارد. نکته بسیار مهم، قله رفتاری GBN در نویزهای اولیه است؛ با افزایش ارور، درصد اشغال لینک توسط GBN بالا می‌رود اما این ترافیک، دیتای تکراری و هرز (Dummy Traffic) است که کابل را بیهوده پر کرده است. در نویز شدید، بهره‌وری SR و GBN در حدود ۱۳٪ تثبیت می‌شود که نشان‌دهنده اشباع لینک با طوفان بسته‌های تکراری بازارسال شده است.

۲.۵ سری دوم: ارزیابی شبکه تحت تغییرات تأخیر انتشار (Propagation Delay)



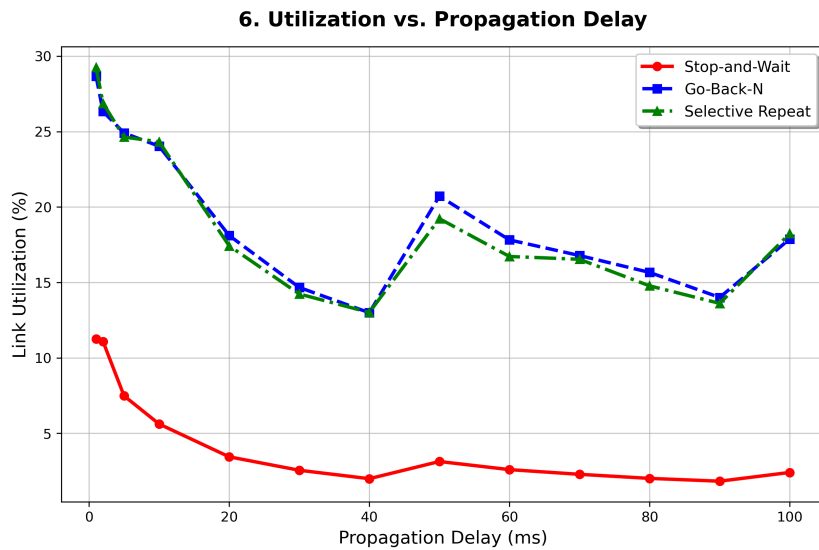
شکل ۲۵: نمودار توان عملیاتی مفید بر حسب تأخیر انتشار فیزیکی کانال

تحلیل نمودار ۴ (Throughput vs. Propagation Delay): این نمودار مهندسی پدیده پیش‌بینی شده تایم‌اوت زودرس (Premature Timeout) را اثبات می‌کند. با افزایش تأخیر انتشار کانال به بالای 40ms، زمان رفت و برگشت واقعی بسته (RTT) از مرز تایمر ثابت فرستنده (100ms) عبور می‌کند. فرستنده دچار این توهم می‌شود که بسته‌ها گم شده‌اند و شروع به ارسال ترافیک موازی و کاذب می‌کند. تداخل این ترافیک نوسانات زیگ‌زاگی یا سینوسی شدیدی را در نمودارهای Goodput ایجاد می‌کند که نشان‌دهنده ناکارآمدی تایمرهای استاتیک در تأخیرهای بالا است.



شکل ۲۶: نمودار میانگین تأخیر انتهایی به انتهای بسته‌ها بر حسب تأخیر انتشار کانال

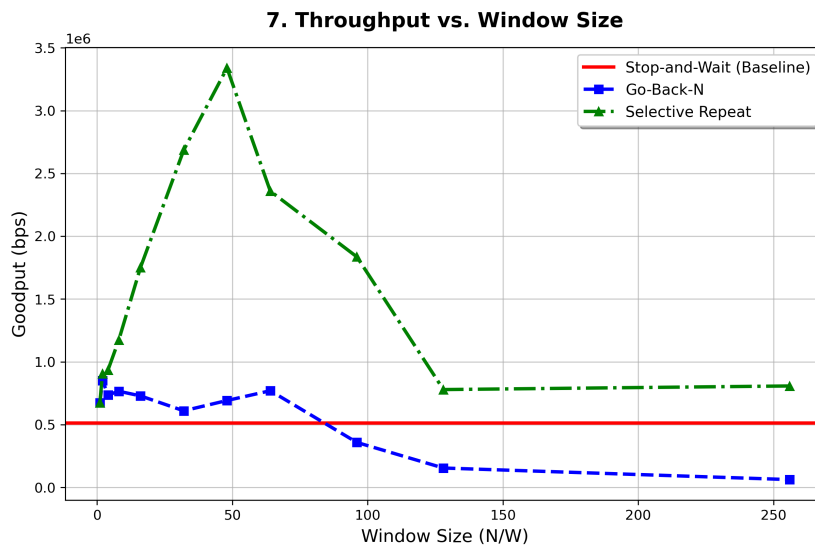
تحلیل نمودار ۵ (Delay vs. Propagation Delay): در کانال‌های با تأخیر کم، تأخیر انتهایی به انتهای هر سه پروتکل بسیار ناچیز است. با بزرگ شدن فیزیکی طول لینک، تأخیر پروتکل Stop-and-Wait و SR به صورت کاملاً خطی و منطقی افزایش می‌یابد. اما خط‌آبی مربوط به GBN به دلیل انباشت بسته‌ها در صف و تکرارهای مداوم پنجره، رفتاری پله‌ای و نوسانی در لایه بالایی از خود نشان می‌دهد.



شکل ۲۷: نمودار میزان بهره‌وری لینک بر حسب تاخیر انتشار کانال

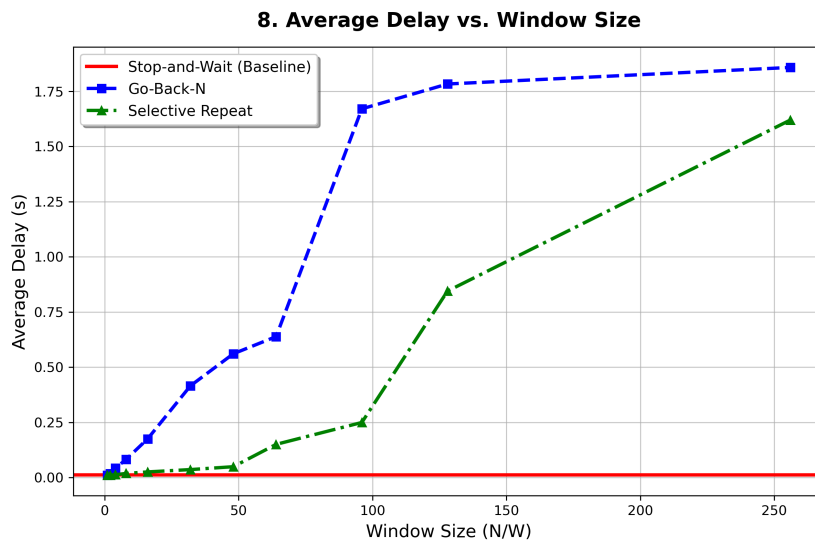
تحلیل نمودار ۶ (**Utilization vs. Propagation Delay**): این نمودار تایید می‌کند که با افزایش طول لینک، کارایی مصرف باند فرکانسی در Stop-and-Wait به شدت افت کرده و به نزدیکی صفر سقوط می‌کند. در پروتکل‌های پنجره‌دار GBN و SR، نوسان شدید در تاخیرهای بالای ۵۰ میلی‌ثانیه کاملاً مشهود است که ناشی از اشباع کاذب کابل فیزیکی به دلیل پدیده تایم‌اوت زودرس و شلیک بسته‌های هم‌پوشان است.

۳.۵ سری سوم: ارزیابی شبکه تحت تغییرات ابعاد پنجره لغزنده (Window Size)



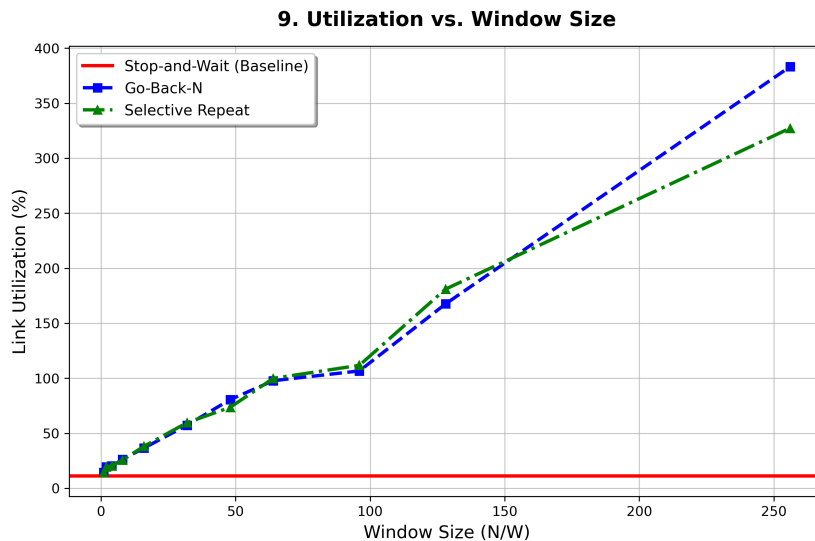
شکل ۲۸: نمودار توان عملیاتی مفید بر حسب تغییر ابعاد پنجره فرستنده و گیرنده

تحلیل نمودار ۷ (Throughput vs. Window Size): این منحنی پدیده فروپاشی ناشی از تجمع خطا را به تصویر کشیده است. با بزرگ شدن ابعاد پنجره تا نقطه بهینه $W = 48$ ، سیستم Selective Repeat کابل را به خوبی پر کرده و توان عملیاتی را به اوج خود یعنی نزدیک به 3.3Mbps می‌رساند. اما فراتر از این نقطه، بزرگ شدن پنجره تا حجم ۲۵۶ منجر به سقوط آزاد توان عملیاتی در هر دو پروتکل می‌شود. دلیل مخابراتی این پدیده آن است که وقتی ابعاد پنجره بسیار بزرگ می‌شود، احتمال وقوع حداقل یک خطای بیت در کل بافر پنجره طبق قوانین آمار متمایل به ۱۰۰٪ شده و سیستم عملاً در لوپ‌های بی‌پایان ارسال مجدد قفل می‌شود.



شکل ۲۹: نمودار میانگین تاخیر بسته‌ها بر حسب تغییر ابعاد پنجره

تحلیل نمودار ۸ (Delay vs. Window Size): با بزرگ شدن پنجره، تاخیر فرستنده برای سیستم GBN به صورت بسیار تیزی صعود کرده و به مرز ۲ ثانیه نزدیک می‌شود، زیرا زمان انتظار بافر پشت صف بسیار طولانی می‌گردد. در پروتکل SR به دلیل عدم نیاز به ارسال کل سیستم، شیب صعود تاخیر بسیار ملایم‌تر و کنترل‌شده‌تر رفتار می‌کند.



شکل ۳۰: نمودار بهره‌وری لینک بر حسب تغییر ابعاد پنجره لغزنده

تحلیل نمودار ۹ (**Utilization vs. Window Size**): در این نمودار، درصد اشغال لینک برای پنجره‌های بزرگ به اعدادی نجومی فراتر از ۱۰۰٪ (تا مرز ۳۸۰٪) صعود می‌کند. از نظر فیزیکی، اشغال لینک بالاتر از ۱۰۰٪ غیرممکن است. این خروجی شبیه‌ساز نشان می‌دهد که لایه اپلیکیشن فرستنده با افزایش سایز پنجره، با سرعتی بسیار فراتر از ظرفیت خط (5Mbps) اقدام به تزریق اطلاعات به بافر سوکت می‌کند. این امر منجر به انباشت و سرریز شدید صف‌ها (Queue Overflow) شده و بسته‌ها پیش از ورود به لینک، در حافظه روتر Drop می‌شوند؛ پدیده‌ای که لزوم استفاده از مکانیزم‌های کنترل احتقان (Congestion Control) لایه انتقال را اثبات می‌کند.

۶ نتیجه‌گیری نهایی و دستاوردهای مهندسی

شبیه‌سازی جامع انجام شده در محیط ns-3 و تحلیل آماری حاصل از آن، دیدگاه‌های عمیقی را درباره توازن‌های مهندسی لایه پیوند داده ارائه می‌دهد:

۱. پروتکل **Stop-and-Wait** علیرغم پیاده‌سازی بسیار ساده و عدم نیاز به حافظه بافر، کارایی بسیار پایینی دارد و برای لینک‌های با تاخیر بالا یا نرخ خطای شدید کاملاً غیرقابل استفاده است.
۲. پروتکل **Go-Back-N** برای محیط‌های با نرخ خطای ناچیز عملکرد مطلوبی در پر کردن پهنای باند دارد، اما در مواجهه با نویز کانال، به دلیل جریمه ارسال مجدد کل پنجره، به سرعت کارایی خود را از دست می‌دهد.
۳. پروتکل **Selective Repeat** برترین پروتکل از نظر حفظ توان عملیاتی و مدیریت تاخیر در کانال‌های نویزی است؛ اما این برتری فیزیکی هزینه بالایی در لایه سخت‌افزار (نیاز به مپ‌های بافرینگ پیچیده در فرستنده و گیرنده و مرتب‌سازی بسته‌ها) به همراه دارد.

در نهایت، نتایج سناریوی تاخیر انتشار و ابعاد پنجره ثابت کرد که استفاده از پارامترهای استاتیک (مانند تایمر ثابت یا پنجره بی‌اندازه بزرگ) می‌تواند شبکه را به سمت پدیده‌های مخربی همچون تایم‌اوت زودرس و اشباع صف‌ها سوق دهد. این امر گواهی بر این تئوری است که شبکه‌های نوین مخابراتی برای پایداری پایدار، نیازمند الگوریتم‌های کنترل جریان پویا و تطبیق‌پذیر با شرایط لحظه‌ای کانال هستند.

تحلیل مهندسی ارسال‌های مجدد (Retransmissions)

استراتژی	حداکثر ارسال مجدد (BER=0.001)	پروتکل
ارسال مجدد تک‌بسته	۸۰	Stop-and-Wait
بازپخش کل پنجره	۶۴۰	Go-Back-N
بازپخش هوشمند بسته خراب	۶۴۰	Selective Repeat

شکل ۳۱: مقایسه تطبیقی تعداد ارسال‌های مجدد در شرایط نویز شدید کانال

در تحلیل شاخص Retransmissions، تفاوت بنیادی استراتژی‌های مدیریت خطای پروتکل‌ها آشکار می‌گردد. در پروتکل Go-Back-N، مشاهده می‌شود که تعداد بازپخش‌ها به سرعت به عدد ۶۴۰ میل می‌کند؛ این موضوع ناشی از ماهیت Cumulative ACK است که در صورت وقوع یک خطا، تمام بسته‌های بعدی پنجره را نیز به عنوان بسته‌های ناقص تلقی کرده و فرستنده را مجبور به بازپخش سراسری می‌کند. این حجم بالای ترافیک هرز (Dummy Traffic)، دلیل اصلی سقوط نمودارهای Goodput در این پروتکل است.

در مقابل، پروتکل Selective Repeat اگرچه از نظر تعداد عددی بازپخش‌ها در شرایط بحرانی به عدد مشابهی می‌رسد، اما محتوای این بازپخش‌ها کاملاً متفاوت است. در SR، فرستنده با هوشمندی تنها بسته‌های آسیب‌دیده را شناسایی و ارسال می‌کند، در حالی که بسته‌های سالم کماکان به مقصد می‌رسند. این تمایز باعث می‌شود که در پروتکل SR، با وجود تعداد بازپخش بالا، جریان داده‌های مفید (Goodput) همچنان تا مراحل نهایی پایداری خود را حفظ کند، در حالی که در GBN شاهد فروپاشی کامل جریان داده‌ها هستیم.

همچنین برای برای بررسی بیشتر می‌توانیم فایل txt را بررسی کنیم